

EXPRESSJS GUIDE

Difference between res.json vs res.send vs res.end in Express.js

Usage of `res.json()` vs `res.send()` vs `res.end()` When and Which one to use?



Punitkumar Harsur · [Follow](#)

Published in Js Gist

4 min read · Dec 25, 2019



Listen



Share

Express is one of the most popular web frameworks for Node.js — probably used by most Node.js developers. It's an excellent framework that has a myriad of HTTP utility methods, and it also performs well.

When working with Express, we get access to a request and a response object, and we can use the latter to send some response back to the requester.

There are a bunch of methods available for us such as `res.json()`, `res.send()` and `res.end()`. The question is, of course, are these different, if so how. In this article, we'll review these differences.

Response

Whenever an Express application server receives an HTTP request, it will provide the developer with an object, commonly referred to as `res`. Here's a really simple example:

```
app.get('/api/test', (req, res) => {  
  // ... do something ...  
});
```

The `res` object is an enhanced version of the response object found in Node.js.

Sending a response

There are multiple ways to send responses using Express, and we'll go through a few examples now.

1. `res.send()`

Sending a response can be achieved by calling the `res.send()` method.

The signature of this method looks like this: `res.send([body])` where the body can be any of the following: `Buffer`, `String`, an `Object` and an `Array`.

This method automatically sets the `Content-Type` response header as well based on the argument passed to the `send()` method, so for example if the `[body]` is a `Buffer` the `Content-Type` will be set to `application/octet-stream` unless of course we programmatically set it to be something else.

Programmatically setting the `Content-Type` header is possible via the `set()` method on the `res` object:

```
res.set('Content-Type', 'text/html');
```

Many developers out there are using Express to create RESTful APIs, and most of the time such APIs return JSON data. Now the question is of course, should we use `res.send()` to return JSON data - since this method accepts objects as part of its argument - or should we use `res.json()`?

Let's see what happens when we try to send JSON data:

```
app.get('/api/test', (req, res) => {  
  res.send({ hello: 'world' });  
});
```

The response header looks like this:

```
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 17
ETag: W/"11-IkjuL6CqqtMReFMfkKvwC0sKj04"
Date: Fri, 10 Aug 2018 09:34:13 GMT
Connection: keep-alive
```

We can see that Express has correctly set the 'Content-Type'. So if this works, why would Express expose `res.json()` at all?

2. `res.json()`

It sends a JSON response. This method is identical to `res.send()` when an object or array is passed, but it also converts non-objects to json.

Namely, it can format the returned JSON data by applying two options:

```
app.set('json replacer', replacer); // property transformation rules
app.set('json spaces', 2); // number of spaces for indentation
```

These two options are collected and passed to the `JSON.stringify()` method: since its signature looks like this: `JSON.stringify(object, replacer, space)`. Once this method is called, the `res.json()` method will then call `res.send()` as well under the hood.

There is no actual difference between `res.send` and `res.json`, both methods are almost identical. `res.json` calls `res.send` at the end. When you have an object or array which you have to pass as a response then you can use any one of them.

But the main difference between `res.json` and `res.send` comes into picture when you have to pass non objects as a response. `res.json` will convert **non objects** (ex. null, undefined etc) as well which are actually not a valid JSON whereas `res.send` will not convert them.

`res.json` also uses json replacer and json spaces application settings.

json replacer : It is a setting which is passed to `JSON.stringify()` for allowing you to manipulate and filter the JSON response.

***json spaces** : Its is a setting which enables the developer to decide if `res.json()` responds with nicely formatte JSON or compact JSON.*

Last but not least let's take a look at `res.end()` . Do we need to call it?

Take a look at this article and see what happens when you don't call `res.end()` : [Concurrent HTTP connections in Node.js](#)

3. `res.end()`

This method kind of makes sense, right? There's a response and once we have collected the data or did something else we want to present that to the caller and as a last step we should end the session — this could be achieved by calling `res.end()` . But do we need it? The answer is two-fold: yes and no.

We can use `res.end()` if we want to end the response **without** providing any data. This could be useful for a 404 page, for example:

```
res.status(404).end();
```

In the code above we are explicitly setting the HTTP status code and immediately after that ending the response.

But what if we want to send some data **and** end the response? The answer is simple, `res.send()` (and remember, `res.json()`) both allow us to send some data **and** they also end the response, so there's no need to explicitly call `res.end()` .

Please also note that `res.end()` does not send an ETag header for the HTTP response whereas `res.send()` does. This Entity Tag header is important when working with web cache validation - it helps caches to be more efficient as well as saves bandwidth.

Conclusion

To sum things up, `res.json()` allows for extra formatting of the JSON data - if this is not required `res.send()` can also be used to return a response object using Express. Both of these methods also end the response correctly, and there's no further action required.